

Dossier technique - Insertion média & Rollback

Ce document détaille ligne par ligne les fonctions de `ProjectInsertion` et `ProjectRollback`, et explique comment l'UI dans `ProjectWidget` les utilise.

Fichiers clés

- `ui/projectinsertion.h` / `ui/projectinsertion.cpp`
 - `ui/projectrollback.h` / `ui/projectrollback.cpp`
 - `ui/projectwidget.cpp` (parties insertion + rollback)
-

`ProjectInsertion` — explication fonctionnalité par fonctionnalité

Fichier: `ui/projectinsertion.h`

- Déclare la classe `ProjectInsertion` et les méthodes publiques:
 - `ProjectInsertion()` (constructeur)
 - `~ProjectInsertion()` (destructeur)
 - `loadMediaListFromFile()` : retourne `QStringList` de chemins médias
 - `saveMediaListToFile(const QStringList &mediaPaths)` : écriture
 - `getMediaListFilePath()` : chemin de fichier principal
 - `getMediaListFilePathForProject(qint64 projectId)` : chemin fichier par projet
 - `initializeFilePath()` : méthode privée pour initialiser `mediaListFilePath`

Fichier: `ui/projectinsertion.cpp` — décomposition

1. Constructeur / Destructeur

```
ProjectInsertion::ProjectInsertion()
{
    initializeFilePath();
}

ProjectInsertion::~~ProjectInsertion()
{
}
```

- Ligne 1–3: le constructeur appelle `initializeFilePath()` pour définir `mediaListFilePath`. Bonne pratique: centraliser le chemin ici pour éviter la duplication.
- Le destructeur n'a rien à faire (pas de ressources allouées explicitement).

2. initializeFilePath()

```

void ProjectInsertion::initializeFilePath()
{
    QString dataPath = QDir::currentPath();
    if (!QDir(dataPath).exists()) {
        QDir().mkpath(dataPath);
    }
    mediaListFilePath = QDir(dataPath).filePath("media_paths.txt");
    qDebug() << "ProjectInsertion::initializeFilePath() - Media list file
path:" << mediaListFilePath;
}

```

- `QDir::currentPath()` : renvoie le répertoire courant (là où l'exécutable est lancé). Attention: pour des installations multi-users, préférer `QStandardPaths::writableLocation(QStandardPaths::AppDataLocation)`.
- `mkpath(dataPath)` crée le répertoire si nécessaire.
- `mediaListFilePath` sera donc quelque chose comme `C:/.../media_paths.txt`.
- `qDebug()` affiche le chemin dans les logs pour faciliter le debugging.

3. loadMediaListFromFile()

```

QStringList ProjectInsertion::loadMediaListFromFile()
{
    QStringList result;
    QFile file(mediaListFilePath);

    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "ProjectInsertion::loadMediaListFromFile() - File does not
exist yet:" << mediaListFilePath;
        return result;
    }

    QTextStream in(&file);
    while (!in.atEnd()) {
        QString line = in.readLine().trimmed();
        if (!line.isEmpty()) {
            result.append(line);
        }
    }

    file.close();
    qDebug() << "ProjectInsertion::loadMediaListFromFile() - Loaded" <<
result.count() << "media paths";
    return result;
}

```

- Ouvre le fichier en lecture texte.
- `QTextStream` lit ligne par ligne. Chaque ligne correspond à un chemin vers un fichier média.
- `trimmed()` Élimine espaces/retours inutiles.

- Lignes vides sont ignorées.
- Retourne la liste de chemins (chemins en string, non d'objets QFileInfo). Le code ne gère pas JSON ni métadonnées : simple liste texte.

4. saveMediaListToFile()

```
void ProjectInsertion::saveMediaListToFile(const QStringList &mediaPaths)
{
    QFile file(mediaListFilePath);

    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qDebug() << "ProjectInsertion::saveMediaListToFile() - Failed to open
file for writing:" << mediaListFilePath;
        return;
    }

    QTextStream out(&file);
    for (const QString &path : mediaPaths) {
        out << path << "\n";
    }

    file.close();
    qDebug() << "ProjectInsertion::saveMediaListToFile() - Saved" <<
mediaPaths.count() << "media paths";

    // If saved list is empty then delete the file to keep project root clean
    if (mediaPaths.isEmpty()) {
        if (QFile::exists(mediaListFilePath)) {
            QFile::remove(mediaListFilePath);
            qDebug() << "ProjectInsertion::saveMediaListToFile() - Removed
empty file:" << mediaListFilePath;
        }
    }
}
```

- Ouvre le fichier en écriture (écrase l'ancien contenu). Si l'ouverture échoue, affiche un message et retourne.
- Écrit chaque chemin sur une nouvelle ligne.
- Si la liste est vide, supprime le fichier (nettoyage automatique).
- Amélioration possible: utiliser `QSaveFile` pour écrire de façon atomique (prévenir fichiers corrompus en cas de crash pendant l'écriture).

5. getMediaListFilePath()

```
QString ProjectInsertion::getMediaListFilePath() const
{
    return mediaListFilePath;
}
```

- Simple getter, utile pour des tâches de nettoyage (UI supprime persistences si besoin).

6. getMediaListFilePathForProject(qint64 projectId)

```
QString ProjectInsertion::getMediaListFilePathForProject(qint64 projectId)
const
{
    QString dataPath = QDir::currentPath();
    return
    QDir(dataPath).filePath(QString("media_paths_project_%1.txt").arg(projectId));
}
```

- Retourne un nom de fichier par projet si on préfère stocker la liste par projet. L'UI n'utilise pas toujours cette méthode mais elle est utile si on veut conserver des médias liés seulement à un projet spécifique.

Comment l'UI utilise ProjectInsertion

- ProjectWidget::loadMediaListUI() appelle insertionHandler->loadMediaListFromFile() pour remplir ui->insertionList.
- ProjectWidget::onInsertionImportClicked() appelle saveMediaListToFile(currentList) pour sauvegarder la liste après import.
- ProjectWidget::onInsertionRemoveClicked() et onInsertionClearClicked() appellent aussi saveMediaListToFile() (ou suppriment le fichier) pour mettre à jour la liste persistante.

ProjectRollback — explication fonction par fonction (ligne par ligne)

Fichier: ui/projectrollback.h

- Interface simple exposant 5 points d'entrée: loadRollbackHistoryFromFile, saveRollbackHistoryToFile, addAction, rollbackLastAction, getRollbackHistoryFilePath.

Fichier: ui/projectrollback.cpp — décomposition

1. Constructeur/Destructeur

```
ProjectRollback::ProjectRollback()
{
    initializeFilePath();
}

ProjectRollback::~~ProjectRollback()
{
}
```

- Le constructeur appelle `initializeFilePath()`.

2. initializeFilePath()

```
void ProjectRollback::initializeFilePath()
{
    QString dataPath = QDir::currentPath();
    if (!QDir(dataPath).exists()) {
        QDir().mkpath(dataPath);
    }
    rollbackHistoryFilePath = QDir(dataPath).filePath("rollback_history.txt");
    qDebug() << "ProjectRollback::initializeFilePath() - Rollback history file
path:" << rollbackHistoryFilePath;
}
```

- Même pattern que `ProjectInsertion` : crée le dossier si nécessaire et définit `rollbackHistoryFilePath`.

3. loadRollbackHistoryFromFile()

```
QStringList ProjectRollback::loadRollbackHistoryFromFile()
{
    QStringList result;
    QFile file(rollbackHistoryFilePath);

    if (!file.open(QIODevice::ReadOnly | QIODevice::Text)) {
        qDebug() << "ProjectRollback::loadRollbackHistoryFromFile() - File does
not exist yet:" << rollbackHistoryFilePath;
        return result;
    }

    QTextStream in(&file);
    while (!in.atEnd()) {
        QString line = in.readLine().trimmed();
        if (!line.isEmpty()) {
            result.append(line);
        }
    }

    file.close();
    qDebug() << "ProjectRollback::loadRollbackHistoryFromFile() - Loaded" <<
result.count() << "history entries";
    return result;
}
```

- Même logique que la lecture média : retourne une liste d'actions.
- Chaque ligne contient une représentation textuelle d'une action: ex: "Added project: 123 - Titre".

4. saveRollbackHistoryToFile(const QStringList &history)

```
void ProjectRollback::saveRollbackHistoryToFile(const QStringList &history)
{
    QFile file(rollbackHistoryFilePath);

    if (!file.open(QIODevice::WriteOnly | QIODevice::Text)) {
        qDebug() << "ProjectRollback::saveRollbackHistoryToFile() - Failed to
open file for writing:" << rollbackHistoryFilePath;
        return;
    }

    QTextStream out(&file);
    for (const QString &entry : history) {
        out << entry << "\n";
    }

    file.close();
    qDebug() << "ProjectRollback::saveRollbackHistoryToFile() - Saved" <<
history.count() << "history entries";

    // Remove file if history is empty
    if (history.isEmpty()) {
        if (QFile::exists(rollbackHistoryFilePath)) {
            QFile::remove(rollbackHistoryFilePath);
            qDebug() << "ProjectRollback::saveRollbackHistoryToFile() - Removed
empty file:" << rollbackHistoryFilePath;
        }
    }
}
```

- Écrit l'historique ligne par ligne. Si l'historique est vide, supprime le fichier pour ne pas laisser de trace.
- Idem à **ProjectInsertion** — peut être amélioré en Queue + QSaveFile.

5. addAction(const QString &action, const QStringList ¤tHistory)

```
void ProjectRollback::addAction(const QString &action, const QStringList
&currentHistory)
{
    QStringList updatedHistory = currentHistory;
    updatedHistory.append(action);
    saveRollbackHistoryToFile(updatedHistory);
    qDebug() << "ProjectRollback::addAction() - Added action:" << action;
}
```

- Reconstitue la liste courante localement, y ajoute la nouvelle action, puis réécrit le fichier. C'est simple mais comporte un double parcours: UI lit -> UI envoie liste -> addAction écrit.

- Amélioration: `addAction` pourrait ouvrir le fichier en mode `Append` et écrire juste la nouvelle ligne, évitant d'écrire toute la liste à chaque ajout.
- Exemple d'action: `QString action = QString("Added project: %1 - %2").arg(projectId).arg(title).`

6. `rollbackLastAction(QStringList &history)`

```
QString ProjectRollback::rollbackLastAction(QStringList &history)
{
    if (history.isEmpty()) {
        qDebug() << "ProjectRollback::rollbackLastAction() - History is empty";
        return QString();
    }

    QString lastAction = history.takeLast();
    saveRollbackHistoryToFile(history);
    qDebug() << "ProjectRollback::rollbackLastAction() - Rolled back action:"
    << lastAction;
    return lastAction;
}
```

- Retire la dernière action (LIFO) de la liste et sauvegarde l'état modifié.
- Important: cette méthode n'exécute aucune opération sur la DB, elle se contente de renvoyer la chaîne de la dernière action.
- L'UI `onRollbackDoClicked()` affiche un popup pour confirmer et, si accepté, supprime la ligne côté UI et appelle cette méthode pour mettre à jour le fichier.

7. `getRollbackHistoryFilePathForProject(qint64 projectId)`

```
QString ProjectRollback::getRollbackHistoryFilePathForProject(qint64 projectId)
const
{
    QString dataPath = QDir::currentPath();
    return
    QDir(dataPath).filePath(QString("rollback_history_project_%1.txt").arg(projectI
    d));
}
```

- Permet un stockage par projet.

Comment l'UI utilise `ProjectRollback`

- `ProjectWidget::loadRollbackHistoryUI()` lit les entrées et les met dans `ui->rollbackList`.
- `onRollbackAddClicked()` récupère la nouvelle entrée depuis `ui->rollbackNewEntryEdit` et appelle `rollbackHandler->addAction(text, currentHistory)`.
- `onRollbackDoClicked()` demande confirmation, appelle `rollbackHandler->rollbackLastAction(currentHistory)` et supprime l'entrée de l'UI.

- `onRollbackClearClicked()` efface tout.

Tests manuels recommandés

1. Ouvrir la page Insertion — importer quelques images/vidéos — voir apparaître dans `ui->insertionList` — fermer et rouvrir l'app pour vérifier la persistance.
2. Ajouter des actions à Rollback, fermer et rouvrir l'app, vérifier que la liste persiste.
3. Tester `rollback` en ajoutant une action textuelle (`Added project: 123`) puis appeler `rollback` — vérifiez que `rollback` renvoie cette chaîne.

Améliorations et meilleures pratiques (recommandées)

1. Ecriture atomique et sûre : utiliser `QSaveFile` (Qt) pour sauvegarder des fichiers de configuration :

```
QSaveFile file(filePath);
if (file.open(QIODevice::WriteOnly)) {
    QTextStream out(&file);
    // ... write
    file.commit();
}
```

Cela évite de corrompre le fichier si l'app part en crash pendant l'écriture.

2. Stockage AppData : pour être compatible OS, utilisez `QStandardPaths::writableLocation(QStandardPaths::AppDataLocation)` comme répertoire de données au lieu de `QDir::currentPath()`.
3. Format structuré : stocker l'historique dans un format JSON (via `QJsonDocument`) plutôt qu'une simple ligne texte — cela permet d'encoder des métadonnées (timestamp, type d'action, userId, undo SQL, etc.).
4. Rollback réel : conserver une représentation sérialisée de l'opération inverse (ex: insertion->INSERT, rollback->DELETE) et exécuter la requête SQL correspondante sur confirm. Ex: conserver `{"op":"INSERT","id":123,"table":"PROJECTS"}` et exécuter `DELETE FROM PROJECTS WHERE ID_PROJECT=123`.
5. Atomicité/Transactions : pour rollback DB, envelopper les opérations dans une transaction si les modifications affectent plusieurs tables.
6. Multithreading / multi-utilisateur: Ajoutez un mécanisme de verrouillage (verrou DB, verrou local) pour éviter les états incohérents si deux utilisateurs modifient la liste simultanément.

Exemple de format JSON suggéré pour l'historique (par entrée) :


```
{
  "timestamp": "2025-11-19T21:34:00Z",
  "action": "ADD_PROJECT",
  "projectId": 123,
  "title": "Mon Projet",
  "userId": 42,
  "undoSql": "DELETE FROM PROJECTS WHERE ID_PROJECT = 123"
}
```

- `undoSql` peut être exécuté par l'UI après confirmation pour effectuer un rollback réel.

Annexes: commandes pour tests & debug

- Lister le fichier de média: `Get-Content .\media_paths.txt` (PowerShell)
- Lister l'historique rollback: `Get-Content .\rollback_history.txt`

Si vous voulez que je transforme `ProjectRollback` en un système de rollback réel (exécution d'`undoSql`) et que j'intègre `QSaveFile` et `QStandardPaths`, je peux l'implémenter et fournir tests unitaires. Voulez-vous que je le fasse ?